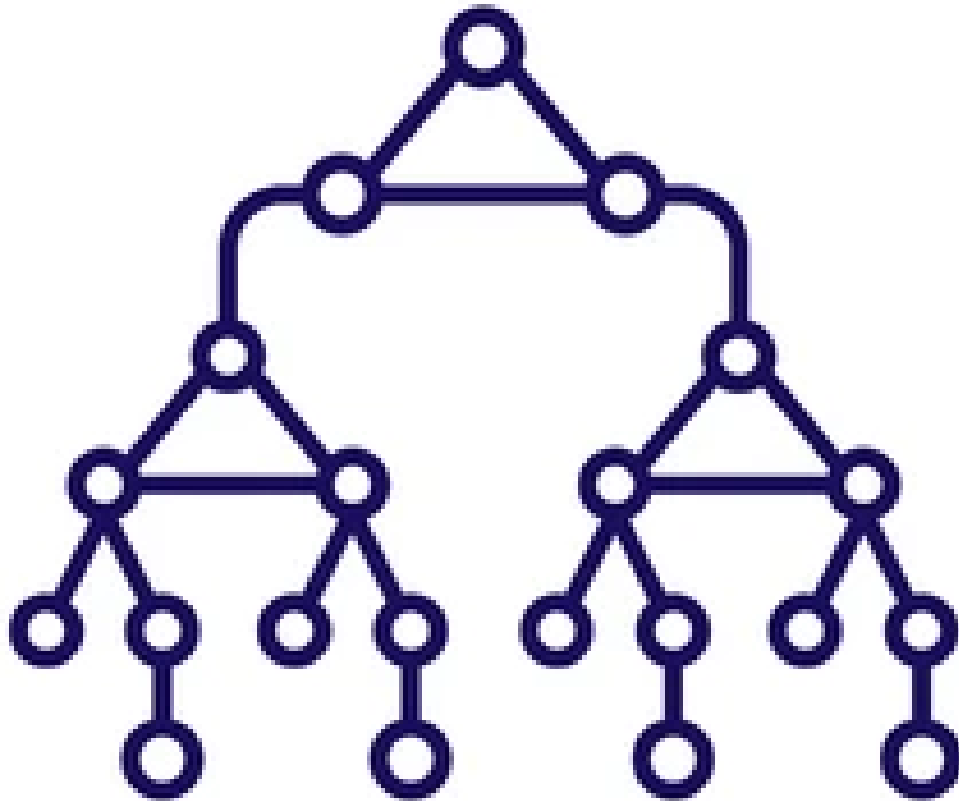


ACTIVIDAD 4

Estructura de datos

Docente: Alberto Lemus Mera



ÍNDICE

Objetivo.....	1
Instrucciones.....	1
Código.....	1
Nodo.java.....	1
Empleado.java.....	2
Arbolbinario.java.....	2
App.java.....	4
Código Corriendo.....	5
¿Cuál y porqué es mejor para este caso Árbol binario o secuencial?.....	5

Objetivo

- Comprender la estructura y funcionamiento de los árboles binarios.
- Implementar y manipular árboles binarios en Java.
- Aplicar los conceptos aprendidos en un caso práctico: gestión de empleados.

Instrucciones

1. **Implementación del árbol binario:**
 - a. Crea una clase **`Nodo`** que represente los nodos del árbol.
 - b. Crea una clase **`ArbolBinario`** que incluya métodos para insertar, eliminar y buscar nodos, así como los diferentes tipos de recorridos.
2. **Realización de operaciones:**
 - a. Inserta elementos en el árbol y muestra el resultado.
 - b. Elimina elementos del árbol y muestra el resultado.
 - c. Realiza búsquedas en el árbol y muestra el resultado.
 - d. Implementa y demuestra los recorridos **preorden**, **inorden** y **postorden**.
3. **Caso práctico: Gestión de empleados**
 - a. Proporciona una lista de empleados con sus identificadores únicos (ID).
 - b. Implementa un sistema donde los empleados se inserten en un árbol binario basado en su ID.
 - c. Realiza operaciones de búsqueda para encontrar empleados por su ID.
 - d. Demuestra cómo el sistema mejora la eficiencia en la gestión de empleados en comparación con una búsqueda secuencial.
4. **Reporte de la actividad:**
 - a. Incluye el código fuente desarrollado.
 - b. Explica el funcionamiento del código.
 - c. Presenta las evidencias de la ejecución del programa (capturas de pantalla o logs).
 - d. Incluye una reflexión sobre la importancia y aplicaciones de los árboles binarios en programación.

Código

Nodo.java

```
public class Nodo {  
    Empleado empleado;  
    Nodo left;  
    Nodo right;  
  
    public Nodo(Empleado empleado) {  
        this.empleado = empleado;  
        this.left = null;  
        this.right = null;  
    }  
}
```

Empleado.java

```
public class Empleado {
    int id;
    String nombre;

    public Empleado(int id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }

    @Override
    public String toString() {
        return "[ID: " + id + "] " + nombre;
    }
}
```

Arbolbinario.java

```
public class Arbolbinario {
    private Nodo raiz;

    public Arbolbinario() {
        this.raiz = null;
    }

    private Nodo insertarRecurativo(Nodo actual, Empleado empleado) {
        if (actual == null) {
            return new Nodo(empleado);
        }
        if (empleado.id < actual.empleado.id) {
            actual.left = insertarRecurativo(actual.left, empleado);
        } else if (empleado.id > actual.empleado.id) {
            actual.right = insertarRecurativo(actual.right, empleado);
        }
        return actual;
    }

    public void insertar(Empleado empleado) {
        raiz = insertarRecurativo(raiz, empleado);
    }

    private Empleado buscarRecurativo(Nodo actual, int id) {
        if (actual == null || actual.empleado.id == id) {
            return actual != null ? actual.empleado : null;
        }
        if (actual.empleado.id > id) {
            return buscarRecurativo(actual.left, id);
        }
        return buscarRecurativo(actual.right, id);
    }

    public Empleado buscar(int id) {

```

```
        return buscarRecursoivo(raiz, id);
    }

    private Nodo eliminarRecursoivo(Nodo actual, int id) {
        if (actual == null) return actual;

        if (id < actual.empleado.id) {
            actual.left = eliminarRecursoivo(actual.left, id);
        } else if (id > actual.empleado.id) {
            actual.right = eliminarRecursoivo(actual.right, id);
        } else {
            if (actual.left == null) return actual.right;
            else if (actual.right == null) return actual.left;

            actual.empleado = valorMinimo(actual.right);
            actual.right = eliminarRecursoivo(actual.right,
actual.empleado.id);
        }
        return actual;
    }

    public void eliminar(int id) {
        raiz = eliminarRecursoivo(raiz, id);
    }

    private Empleado valorMinimo(Nodo actual) {
        Empleado min = actual.empleado;
        while (actual.left != null) {
            min = actual.left.empleado;
            actual = actual.left;
        }
        return min;
    }

    public void inOrdenRecursoivo(Nodo nodo) {
        if (nodo == null) return;
        inOrdenRecursoivo(nodo.left);
        System.out.print(nodo.empleado + " | ");
        inOrdenRecursoivo(nodo.right);
    }

    public void preOrdenRecursoivo(Nodo nodo) {
        if (nodo == null) return;
        System.out.print(nodo.empleado + " | ");
        preOrdenRecursoivo(nodo.left);
        preOrdenRecursoivo(nodo.right);
    }

    public void postOrdenRecursoivo(Nodo nodo) {
        if (nodo == null) return;
        postOrdenRecursoivo(nodo.left);
        postOrdenRecursoivo(nodo.right);
    }
}
```

```
        System.out.print(nodo.empleado + " | ");
    }

    public void mostrarInOrden() {
        System.out.println("InOrden:");
        inOrdenRecursoivo(raiz);
        System.out.println("\n");
    }

    public void mostrarPreOrden() {
        System.out.println("PreOrden:");
        preOrdenRecursoivo(raiz);
        System.out.println("\n");
    }

    public void mostrarPostOrden() {
        System.out.println("PostOrden:");
        postOrdenRecursoivo(raiz);
        System.out.println("\n");
    }
}
```

App.java

```
public class App {
    public static void main(String[] args) {
        Arbolbinario arbol = new Arbolbinario();

        Empleado[] empleados = {
            new Empleado(1, "Erick"),
            new Empleado(2, "Luis"),
            new Empleado(3, "Sofia"),
            new Empleado(4, "Ana"),
            new Empleado(5, "Carlos"),
            new Empleado(6, "Maria"),
            new Empleado(7, "Pedro")
        };

        System.out.println("--- Insertando empleados en el árbol  
---");
        for (Empleado emp : empleados) {
            arbol.insertar(emp);
        }

        System.out.println("\n--- Probando los recorridos ---");
        arbol.mostrarInOrden();
        arbol.mostrarPreOrden();
        arbol.mostrarPostOrden();

        System.out.println("--- Búsqueda de Empleados ---");
        Empleado buscado = arbol.buscar(1);
    }
}
```

```

        System.out.println("Buscando ID 1: " + (buscado != null ?
"Encontrado -> " + buscado.nombre : "No encontrado.));

        buscado = arbol.buscar(2);
        System.out.println("Buscando ID 2: " + (buscado != null ?
"Encontrado -> " + buscado.nombre : "No encontrado.));

        System.out.println("\n--- Eliminación de Empleados ---");
        System.out.println("Eliminando ID 3...");
        arbol.eliminar(3);
        arbol.mostrarInOrden();
    }
}

```

Código Corriendo

```

Estructura de datos/Tarea4 on ? Semana4 [?] via v25.0.4
> java App.java
--- Insertando empleados en el árbol ---

--- Probando los recorridos ---
InOrden:
[ID: 1] Erick | [ID: 2] Luis | [ID: 3] Sofia | [ID: 4] Ana | [ID: 5] Carlos | [ID: 6] Maria | [ID: 7] Pedro |

PreOrden:
[ID: 1] Erick | [ID: 2] Luis | [ID: 3] Sofia | [ID: 4] Ana | [ID: 5] Carlos | [ID: 6] Maria | [ID: 7] Pedro |

PostOrden:
[ID: 7] Pedro | [ID: 6] Maria | [ID: 5] Carlos | [ID: 4] Ana | [ID: 3] Sofia | [ID: 2] Luis | [ID: 1] Erick |

--- Búsqueda de Empleados ---
Buscando ID 1: Encontrado → Erick
Buscando ID 2: Encontrado → Luis

--- Eliminación de Empleados ---
Eliminando ID 3...
InOrden:
[ID: 1] Erick | [ID: 2] Luis | [ID: 4] Ana | [ID: 5] Carlos | [ID: 6] Maria | [ID: 7] Pedro |

```

¿Cuál y porqué es mejor para este caso Árbol binario o secuencial?

Para este caso es mejor el árbol binario por qué hacerlo de manera secuencial implica revisar elemento por elemento o sea si tuviéramos 10,000 empleados en el peor caso se harían 10,000 comprobaciones y en el árbol binario se descarta la mitad del árbol por lo que solo se harían unas 14 comprobaciones